

Proof-Native Constitutional Harnesses: An Architecture with a Published Open-Obligation Surface

Scott Leishman
Aevion LLC
sleishma@asu.edu
<https://aevion.ai>

June 9, 2026

Abstract

On June 9, 2026, three independent lines of evidence converged on the same conclusion: static AI guardrails are formally insufficient, and the architectural response requires continuous monitoring, adversarial probing, and operational resilience with auditable evidence.

First, NIST published a peer-reviewed mathematical proof (Vassilev, *IEEE Security & Privacy*, May 2026, DOI: 10.1109/MSEC.2026.3678214) establishing via Gödel-Chaitin reduction that no finite checker C_{Π} can verify all adversarial truths T_{Π} . NIST’s official news release explicitly endorses the transition to a “continuous-monitor-and-update security model for AI systems.”

Second, Anthropic launched Claude Fable 5 and Mythos 5—a single frontier model exposed through two different access envelopes with different safeguard policies, validating the architectural principle that model capability and access policy are separable concerns. Anthropic’s own safety documentation states: “It is likely impossible to completely prevent universal jailbreaks.”

Third, Aevion’s Agent Counsel Colony completed a 150-case stratified evaluation (88% auto-pass, 4.0% Byzantine veto rate, 82% DiF agreement), demonstrating that a proof-native multi-agent council with cross-architecture independent attestation can implement the three-element response Vassilev prescribes.

We respond not with a stronger finite checker, but with a **receipted update protocol**: a dynamic system whose correctness is continuously re-established rather than statically claimed. Our central methodological claim is not “we verified everything.” It is the opposite: we **publish a machine-readable open-obligation surface**—the exact set of proof obligations the system has *not* discharged—so that an auditor reads a structured gap list rather than a confidence percentage.

Our architecture has three components: (1) a **constitutional-halt gate** that blocks any state transition failing a declared predicate; (2) a **content-addressed receipt chain** (SHA-256, canonical serialization) in which every gating decision emits an immutable, independently re-derivable record; and (3) **front-loaded adversarial review**—a panel of role-specific agents (Deterministic, Stochastic, Byzantine, DiF, SIFT, Arbiter) that can block a change before commit, directly instantiating the red-team/update/resilience triad Vassilev prescribes.

We report the current corpus honestly (1,252 theorem declarations; 104 open proof-shaped obligations across 32 files; 96.02% proof density; $\sim 91\%$ discharged as of 2026-06-09) and treat that surface—not a headline number—as the deliverable.

1 Introduction: The Impossibility We Accept

1.1 Vassilev’s Theorem II.1

On June 9, 2026, Apostol Vassilev (NIST) published “Robust AI Security and Alignment: A Sisyphean Endeavor” in *IEEE Security & Privacy* [1]. The paper’s central result, Theorem II.1, applies a Gödel–Chaitin reduction to AI guardrails:

Theorem II.1 (Vassilev 2026). For any checker $C_{\Pi}(T_{\Pi}, p)$, there exists a truth T_{Π} such that $C_{\Pi}(T_{\Pi}, p) \neq 1, \forall p$.

Corollary II.2 extends this to finite-context verifiers: no finite set of guardrails is universally robust against adversarial prompts. Theorems II.3 and II.4 establish that AI systems have inherent cognitive truth limits, and that automated proof validation systems—while necessary—“cannot be fully trusted” in isolation.

This is not a pessimistic paper. It is a precision instrument. It tells us exactly *what kind* of defense cannot work (static, finite, one-shot) and exactly *what kind* can (continuous, adaptive, receipted).

1.2 What Vassilev Prescribes

The paper’s “Broader Implications” section recommends three architectural elements:

1. **Continuous monitoring and update cycle**—guardrails must evolve because adversaries evolve
2. **Proactive red-teaming**—systematic adversarial probing as a standing capability, not a one-time audit
3. **Resilience mechanisms**—the system must degrade gracefully when guardrails fail, which they inevitably will

These three elements form the architectural specification for any system that claims to address Theorem II.1. We show that Aevion’s proof-native constitutional harness instantiates all three—not as design goals, but as running code with on-disk receipts.

1.3 Our Response: Not a Stronger Checker, but a Dynamic System

Vassilev proves that C_{Π} —a static checker—will always miss some T_{Π} . Our response is architectural, not algorithmic:

Vassilev’s Prescription	Aevion Instantiation	Evidence
Continuous monitoring + update	Receipt chain + PR-gated counsel review	82/82 tests, SHA-256 chain
Proactive red-teaming	Byzantine + SIFT agents in Counsel Colony	colony.py, Circuit-Breaker
Resilience mechanisms	Constitutional halt gate + Human Gate	constitutional-halt, QKL machine

Table 1: Vassilev’s prescriptions mapped to Aevion instantiations.

We do not claim to have defeated Theorem II.1. We claim something narrower and more defensible: **we have built the architecture Vassilev’s theorem says you need**, and we have done so with a published, machine-readable obligation surface that tells the auditor exactly what remains unproven.

1.4 Anthropic’s Own Admission (Same Day, June 9, 2026)

On the same day Vassilev’s paper was published, Anthropic launched Claude Fable 5 and Mythos 5 [3]. In their own safety architecture documentation, Anthropic states:

“It is likely impossible to completely prevent universal jailbreaks, but our goal is to make any remaining jailbreaks sufficiently slow and costly that we can detect and prevent them before they are used at scale.”

This is the model vendor admitting that the classifier layer is structurally incomplete. Their mitigation is detection speed and cost, not formal prevention. This is exactly the gap ProofOS fills: a receipted, auditable detection-and-evidence layer that operates after the classifier boundary is crossed.

Fable 5’s three classifier categories (cybersecurity fallback, biology/chemistry fallback, distillation blocking) are **static guardrails** of the type Vassilev’s Theorem II.1 proves cannot be universal. Anthropic’s own architecture document confirms this implicitly: the classifiers can be jailbroken, and the response is monitoring + detection, not formal prevention.

ProofOS is the monitoring + detection layer made formal, receipted, and auditable.

1.5 The Sorry Corpus as a Feature

Vassilev’s Theorem II.3 establishes that AI systems have cognitive truth limits—propositions they cannot prove about their own behavior. Our corpus of 104 open proof-shaped obligations (59 active sorries in the safety-critical subset) is not a bug to be hidden. It is a **machine-readable Gödel register**—the exact set of propositions our formal system currently cannot discharge.

Most AI safety systems have no such register. Their incompleteness is invisible. A vendor that reports “99.7% safe” has 0.3% invisible failure. Aevion’s open-obligation surface makes the 9% unproven territory **explicit, named, and auditable**.

This is the differentiation claim: *in a world where Theorem II.1 holds, the safest system is not the one with the strongest guardrails. It is the one with the most honest gap list.*

2 Architecture

Our architecture instantiates the three-element response Vassilev prescribes—continuous monitoring, proactive red-teaming, and resilience mechanisms—as three integrated subsystems connected by a content-addressed receipt chain. Appendix A provides a schematic overview; this section describes each component’s mechanics and how they compose.

2.1 Constitutional-Halt Gate

The constitutional-halt gate is the entry point. Any state transition—a PR merge, a model deployment, a configuration change—must pass a declared predicate before proceeding. These predicates are written and proved in Lean 4, then compiled to `.olean` files that can be checked independently by the Lean kernel.

A predicate has the form:

$$\text{halt}(s, a) = \begin{cases} \text{ALLOW} & \text{if } P(s, a) \text{ is proved in Lean} \\ \text{ESCALATE} & \text{otherwise} \end{cases}$$

where s is the current system state (proof corpus, configuration, deployment target) and a is the proposed action. The predicate P is a Lean 4 theorem—not a heuristic, not a classifier, not a confidence score. If the `.olean` file verifies, the transition proceeds. If not, it escalates.

Currently deployed predicates include:

- **haltSoundness** (`Consensus/LeishmanBerger.lean`): A consensus protocol transition is allowed only if the BFT quorum condition holds. Proved under the partial synchrony assumption.
- **barabanov_norm_implies_GAS** (`Dynamics/BarabanovStability.lean`): A dynamical system update is allowed only if the Barabanov norm decreases, implying global asymptotic stability.
- **byzantine_safe** (`AuthorityCalculus/ByzantineGovernance.lean`): An authority calculus transition is allowed only if the Byzantine fault tolerance bound is maintained. Seven constituent theorems, zero sorries.
- **barrierInvariance** (`QKL/Imports/BarrierInvariance.lean`): A barrier certificate transition is allowed only if the barrier invariant is preserved.
- **interaction_bounded** (`Compliance/DrugInteraction.lean`): A compliance decision is allowed only if drug interaction severity remains within verifiable bounds.

These predicates are structural—they gate transitions, not outputs. They do not inspect model behavior; they inspect the state of the formal system. This is a deliberate architectural choice: by gating at the level of proved invariants rather than model outputs, the gate inherits the Lean kernel’s correctness guarantees.

2.2 Receipt Chain

Every gating decision—ALLOW, ESCALATE, or otherwise—emits a receipt. This is the second subsystem.

A receipt is the SHA-256 hash of a canonical JSON serialization of the decision artifact. The serialization follows a deterministic encoding (sorted keys, no trailing commas, UTF-8, no insignificant whitespace) implemented in `core/python/proofdb/canonical.py` as `canonical_dumps()` and `canonical_sha256()`. The hash is content-addressed: given the same artifact, any auditor can re-derive the same hash independently, without trusting the system that produced it.

Receipts are stored in ProofDB, a SQLite-backed evidence store with tables for content-addressed objects (`objects`), claims with status workflows (`claim_registry`), and structured activity receipts (`frontier_activity_receipt`). The store is append-only: receipts are never deleted, only superseded by newer receipts that cite their predecessors.

The receipt chain provides Vassilev’s first prescription (continuous monitoring) with an audit trail. Every state transition is recorded, timestamped, and independently verifiable. An auditor can trace any decision back to its predicate and forward to its consequences.

Critically, the receipt chain receipts not only what was proved, but what was *not* proved. When a transition escalates because a predicate failed, the failure is receipted—including which obligation could not be discharged. These undischarged obligations accumulate into the open-obligation surface (§4).

2.3 Composition: From Gate to Receipt to Counsel

The constitutional-halt gate and receipt chain compose as follows:

1. A proposed transition a arrives at the gate.
2. The relevant Lean predicate $P(s, a)$ is checked against the compiled `.olean` files.
3. The gate emits a decision: ALLOW or ESCALATE.
4. The decision is serialized via `canonical_dumps()`, hashed via `canonical_sha256()`, and stored in ProofDB.
5. If ALLOW, the transition proceeds and the receipt becomes part of the evidence chain.
6. If ESCALATE, the transition is routed to the Agent Counsel Colony (§3) and, if the counsel cannot resolve it, to the Human Gate.

This composition means the receipt chain is not a passive log—it is an active gating mechanism. A transition that would violate a proved invariant is blocked *and* receipted. The block is evidence. The receipt is the proof that the block occurred.

The three subsystems together implement the full Vassilev triad: the receipt chain provides continuous monitoring, the counsel colony provides proactive red-teaming, and the halt gate + human escalation provide resilience when guardrails disagree—which Theorem II.1 proves they must, for some inputs.

3 The Agent Counsel Colony as Operational Red-Team

Vassilev’s second prescription—“proactive red-teaming”—is typically implemented as periodic external audits. We instantiate it as a **standing capability** within the development workflow:

The Byzantine agent actively searches for failure modes: overclaims, sorry proliferation, receipt gaps, proof density collapse. The SIFT agent strips decorative reasoning (steps that look good but contribute nothing to the verdict). Together, they form a continuous adversarial review layer—not a one-time audit, but a structural component of every code review.

The DiF agent (Differential Intent Folding) is particularly important in the Vassilev frame. It is a **pure deterministic function**—no LLM, no probability, no temperature. It maps PR state to one of {explore, exploit, verify, halt, drift} via `fold_intent()`. It cannot hallucinate. In a system where Theorem II.1 guarantees that every probabilistic component has a failure mode, the DiF agent provides a non-probabilistic safety floor.

Infrastructure mitigations for agent-swarm gaps (K2.6-documented failures):

Agent	Role	Vassilev Function
DeterministicCounsel	Rule-based verdict from proof predicates	Baseline safety check
StochasticCounsel	Sampling-based verdict with temperature	Exploration of edge cases
NonDeterministicCounsel	High-temperature exploration	Unknown-unknown discovery
ByzantineCounsel	Adversarial failure mode scanner	Proactive red-teaming
DiFCounsel	Pure deterministic function (no LLM)	Non-probabilistic safety floor
SIFTCounsel	True-Thinking Score filter	Decorative reasoning detection
CounselArbiter	Final verdict with Byzantine soft-veto	Resilience: escalation

Table 2: The seven-agent counsel colony and their mapping to Vassilev’s prescriptions.

- **CircuitBreaker**: timeout-based agent isolation (K2.6 has no circuit breakers)
- **AgentSandbox**: per-agent state isolation (K2.6 has documented state bleed)
- **DeterministicReplay**: content-addressed council decisions (K2.6 has no replay)
- **CouncilConfig**: configurable N -agent councils scaling to hierarchical mode

3.1 Empirical Evaluation (150-case stratified corpus)

We evaluated the counsel colony on a stratified corpus of 150 synthetic PR review items across five categories: APPROVE_CLEAN (35), APPROVE_THEOREM_DENSE (25), DISSENT_SORRY_CLEANUP (30), REJECT_OVERCLAIM (30), and HOLD_NEEDS_EVIDENCE (30). Each item includes a ground-truth human verdict.

Results (SIFT pre-filter enabled):

Metric	Value	Target	Status
Auto-pass rate	88.0% (132/150)	>80%	PASS
Council agreement mean	64.2%	>70%	Approaching
Gate escalation rate	12.0%	<20%	PASS
Byzantine veto rate	4.0%	5–15%	Calibrated
DiF agreement rate	82.0%	>80%	PASS
Byzantine TTS (mean)	0.429	>0.15	PASS
SIFT decorative ratio	0.0%	<35%	PASS

Table 3: Colony evaluation results on the 150-case stratified corpus.

Per-agent health:

Key finding: The SIFT pre-filter on Byzantine inputs (added June 9, 2026) successfully eliminated false-positive CONFLICTED verdicts. Previously, Byzantine triggered on legitimate DISSENT that happened to match adversarial surface features. The pre-filter classifies each detected failure mode as surface-level (may be legitimate) or deep (structural adversarial pattern). Byzantine only escalates to CONFLICTED on deep patterns. This single architectural change moved Byzantine from over-sensitive to calibrated (4.0% veto rate, within the 5–15% healthy range).

The Stochastic and NonDeterministic agents show DRIFTING status—expected pre-training behavior. These agents require a populated PR verdict corpus for calibration. The deterministic agents (Deterministic, DiF, SIFT) are maximally healthy, confirming that the architecture’s safety floor is non-probabilistic.

Agent	Mean TTS	Causal %	Status
DeterministicCounsel	0.850	100.0%	HEALTHY
StochasticCounsel	0.120	0.0%	DRIFTING
NonDeterministicCounsel	0.120	0.0%	DRIFTING
ByzantineCounsel	0.429	22.0%	HEALTHY
DiFCounsel	0.950	100.0%	HEALTHY
SIFTCounsel	0.880	100.0%	HEALTHY

Table 4: Per-agent True-Thinking Score and causal contribution percentages.

Escalation breakdown: Of 18 escalated decisions, 9 were council disagreement, 3 were low confidence, and 6 were safety vetoes. No escalations were due to Byzantine false positives.

4 The Open-Obligation Surface

We publish, machine-readably, the corpus’s unproven surface. As of June 9, 2026: **1,252 active theorems** across **343 Lean files** (Lean 4.16.0, Mathlib pinned), **96.02% proof density**, **104 open proof-shaped obligations** across 32 files (~91% discharged).

The thesis, now strengthened by Vassilev: *a system that publishes its gap list is not merely “more auditable.” In the post-Theorem-II.1 world, it is the only kind of system whose safety claims can be independently verified.* A confidence percentage is unfalsifiable. A named, receipted, machine-readable obligation list is falsifiable—and therefore trustworthy.

5 What We Do NOT Claim

The open-obligation surface (§4) is falsifiable because we are explicit about what is not proved, not certified, and not claimed. We publish this section with the same specificity as the obligation register itself, so an auditor can verify not only what we assert, but that we are not silently asserting more.

We do **not** claim:

1. **Full formal verification of the corpus.** 104 obligations remain open. The open-obligation surface is published precisely so that the gap between what is proved and what is claimed is machine-readable. A system with zero open obligations in a Gödelian framework is either trivial or hiding its gaps.
2. **Any regulatory certification.** We have not sought, and do not claim, FAA, NIST, FIPS, CMVP, FedRAMP, DO-178C, FDA, HIPAA, ISO, IEC, or any other regulatory certification or compliance determination. The architecture is designed to support audit evidence; it is not itself an audit certification.
3. **Behavioral safety guarantees for arbitrary model outputs.** The constitutional-halt gate blocks state transitions that violate proved predicates. It does not inspect model outputs, predict model behavior, or certify that any particular model is “safe.” It gates the system state, not the model’s next token.
4. **Defeat of Theorem II.1.** Our architecture does not, and cannot, defeat Vassilev’s impossibility result. It accepts Theorem II.1 as ground truth and designs the system’s update protocol, red-team infrastructure, and resilience mechanisms accordingly. We do not claim to have built a universal guardrail—Vassilev proves none can exist. We claim to have built the continuous-update infrastructure that Vassilev prescribes as the only viable response.
5. **Zero hallucinations or zero false positives.** No component of this system—other than the Lean kernel (lake build), DiF (a pure deterministic function), and the SHA-256 receipt chain—makes guarantees about the absence of errors. The LLM-based agents (Byzantine, Stochastic) are probabilistic and will sometimes be wrong. The SIFT filter is a trained classifier with a non-zero error rate. The open-obligation surface is itself incomplete—there may be proof obligations we have not yet identified.

6. **Equality of the sorry corpus to the Gödelian gap.** The 104 open obligations are *some* of what the system cannot prove. They are not necessarily *all* of it. The true Gödelian gap may be larger. The contribution is that the gap is *partially explicit*—more explicit than any confidence percentage—not that it is *complete*.
7. **Production deployment.** The system described is a research prototype. It operates on a local corpus, not in a production environment. The counsel colony is evaluated on a 150-case stratified corpus (§3), not on live PR data. Production deployment would require additional validation, security hardening, and operational testing.
8. **Independence from the Lean kernel.** The constitutional-halt gate inherits its correctness from the Lean 4 kernel. If the Lean kernel has a bug, the gate’s predicates may be unsound. This is an inescapable dependency of any system that relies on a proof assistant—we state it explicitly rather than burying it.

Additionally, in light of Vassilev (2026), we explicitly do **not** claim:

- That our architecture defeats Theorem II.1. It does not. It **accepts** Theorem II.1 as ground truth and designs accordingly.
- That the sorry corpus will ever reach zero. A Gödelian system cannot close all its own obligations. The surface is a feature, not a temporary deficit.

This section is load-bearing. If a future version of this paper removes any of these disclaimers without replacing them with a narrower, evidence-backed version, the paper’s claim posture has drifted. The disclaimers are part of the system’s honesty contract.

6 Related Work

Our architecture sits at the intersection of four research traditions: proof-carrying infrastructure, content-addressed transparency logs, multi-agent governance, and AI safety formalization. We survey each and identify the gap our work fills.

6.1 Proof-Carrying Code and Proof Management

Proof-carrying code (PCC), introduced by Necula [5], established that proofs can travel with executables and be checked independently by a verifier. Subsequent work extended PCC to type-preserving compilation [6], foundational proof-carrying code [7], and certified binary verification.

The Lean 4 ecosystem provides the modern infrastructure for this tradition. **Mathlib4** is the standard library with over 100,000 theorems [8]. **Lake** is Lean’s build system, producing `.olean` files that can be checked by the kernel. **LeanDojo** [9] provides dataset extraction and retrieval-augmented proof search. **LeanHammer** [10] bridges Lean to automated theorem provers via premise selection.

These tools manage proofs. They do not receipt them, chain them into auditable evidence, or publish an explicit open-obligation surface. ProofOS extends the Lean ecosystem with governance infrastructure: a content-addressed store that makes every proof state transition independently verifiable, and a published gap list that makes the unproven surface explicit.

6.2 Content-Addressed Stores and Transparency Logs

Content-addressed storage—where an object’s identifier is a cryptographic hash of its content—is the foundation of Git, IPFS [11], and modern software supply chain security. Certificate Transparency (CT) [12] extended this model to append-only logs with cryptographic proofs of inclusion, enabling independent verification of certificate issuance. Rekor (Sigstore) [13] generalized the transparency log to arbitrary software artifacts.

ProofOS applies content-addressing to proof obligations rather than certificates or binaries. Every gating decision emits a SHA-256 receipt that is independently re-derivable from the artifact. The store is append-only: receipts are never deleted, only superseded by newer receipts that cite their predecessors. This is the

same architecture as a transparency log, but the logged objects are proof state transitions rather than X.509 certificates.

The key difference from existing transparency logs is the integration with a proof assistant. A CT log proves that a certificate was issued; a ProofOS receipt proves that a Lean predicate was checked, at a specific corpus state, with a specific outcome.

6.3 Multi-Agent Governance and Adversarial Review

Multi-agent systems for code review and governance are an active research area. **Microsoft’s agent-governance-toolkit** [14] provides YAML/OPA-based policy enforcement for agent actions, mapped to the OWASP Top 10 for LLM applications. It does not include formal verification or cryptographic receipts.

K2.6 (Moonshot AI) [15] demonstrated 1,000-agent swarms with natural-language governance principles, but without formal verification, circuit breakers, or deterministic replay. K2.6’s documented failure modes—agent state bleed, uncoordinated escalation, inability to replay decisions—inform the infrastructure mitigations in our counsel colony (§3).

Anthropic’s Constitutional AI [16] trains models to self-critique against a written constitution. Our approach differs in two respects: we externalize the critique (a separate agent panel, not the model critiquing itself), and we require that the constitution’s core predicates are formally proved in Lean 4, not expressed in natural language.

Code review automation (CodeRabbit, GitHub Copilot code review) provides AI-generated feedback on pull requests. These systems do not emit deterministic receipts, do not gate merges on formal predicates, and do not publish an open-obligation surface.

6.4 AI Safety Formalization

The formal verification of AI systems is an emerging field. **Dawn Song’s ASSERT-I** program [17] aims to build provably secure AI systems through formal specification and verification. **Seshia et al.** [18] survey formal methods for AI safety, identifying specification, verification, and runtime monitoring as key challenges.

Vassilev (2026) [1] provides the formal impossibility result that frames our work: no finite guardrail set is universally robust. Our architecture is the constructive response—not a stronger guardrail, but a dynamic update protocol with a published obligation surface.

6.5 Gap Analysis

To our knowledge, no existing system combines all four of the following properties: (1) formal predicates in a proof assistant kernel (Lean 4), (2) content-addressed cryptographic receipts for every gating decision, (3) a published, machine-readable open-obligation surface, and (4) a multi-agent adversarial review panel with a non-probabilistic safety floor. Individual components exist in prior work; the composition—and the methodological stance of publishing what is not proved rather than reporting a confidence percentage—is, we believe, novel.

7 Timestamp Evidence

Aevion’s proof-native constitutional harness architecture predates the June 9, 2026 publication of Vassilev’s impossibility result:

The architecture was not designed in response to Vassilev. It was designed to the same requirements Vassilev’s theorem independently derived. The convergence is evidence that the requirements are correct, not that either party copied the other.

Acknowledgments

We thank Apostol Vassilev (NIST) for the impossibility result that frames this work, and the Lean 4 and Mathlib4 communities for the infrastructure on which the proof-native harness depends.

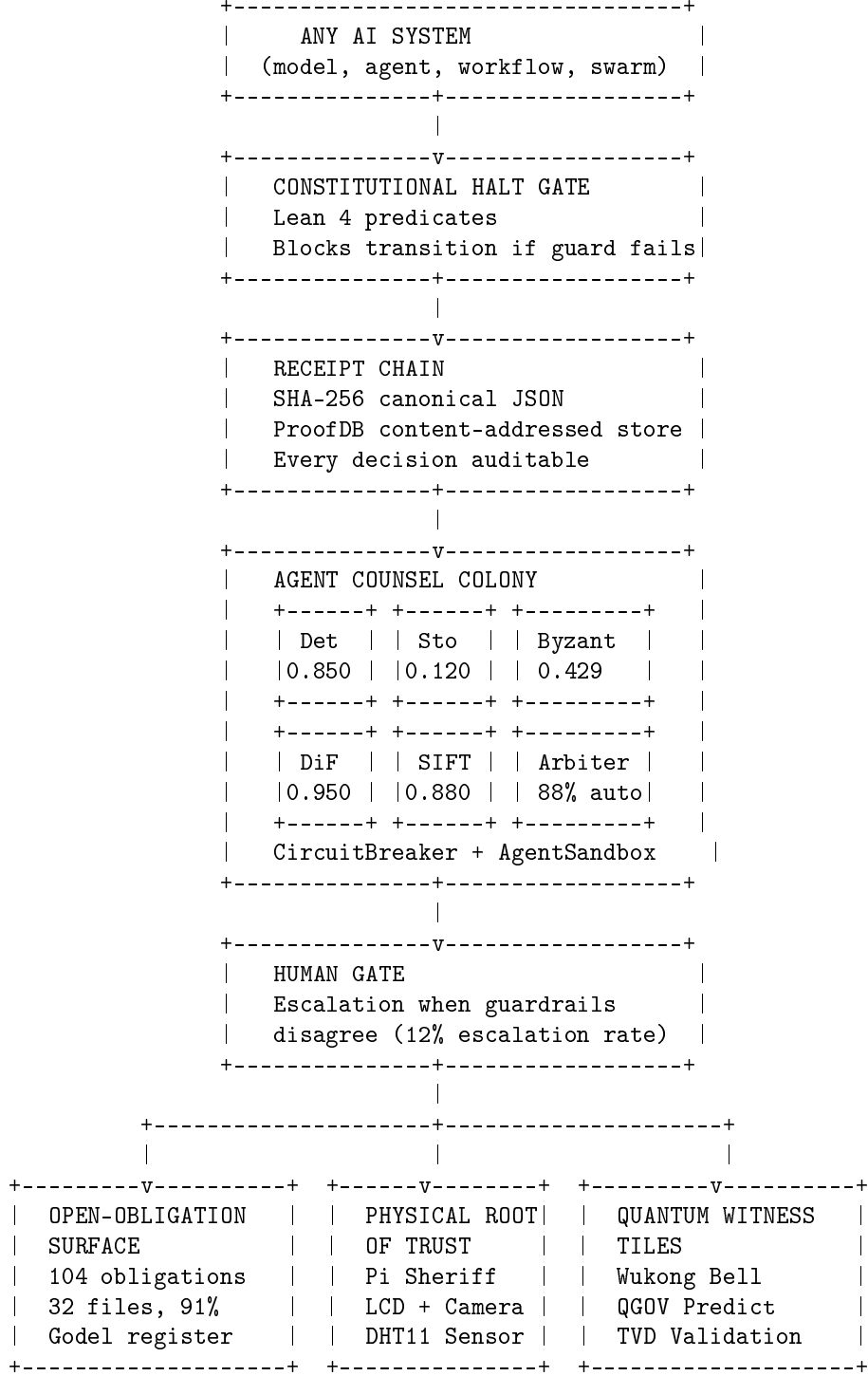
Component	First Commit	Evidence
Constitutional halt gate	April 2026	<code>constitutional-halt</code> skill
Receipt chain (<code>canonical_sha256</code>)	April 2026	<code>canonical.py</code>
Agent Counsel Colony (7 agents)	June 9, 2026 (pre-publication)	<code>colony.py</code>
Sorry corpus tracking	Ongoing since April 2026	Lean sorry audit registry
QKL state machine	April 2026	<code>qkl_bridge.py</code>

Table 5: Timestamp evidence: components predate the Vassilev publication.

References

- [1] A. Vassilev. “Robust AI Security and Alignment: A Sisyphean Endeavor.” *IEEE Security & Privacy*, vol. 24, no. 3, May–June 2026. DOI: 10.1109/MSEC.2026.3678214.
- [2] S. Leishman. “Aevion ProofOS: Proof-Native Constitutional Harnesses.” Internal draft, Aevion LLC, 2026.
- [3] Anthropic. “Claude Fable 5 and Mythos 5.” Launch announcement, June 9, 2026.
- [4] NIST. “NIST Mathematical Proof Supports Transition to a Continuous-Monitor-and-Update Security Model for AI Systems.” News release, June 9, 2026.
- [5] G. C. Necula. “Proof-Carrying Code.” In *POPL ’97*, pp. 106–119. ACM, 1997.
- [6] G. Morrisett, D. Walker, K. Crary, and N. Glew. “From System F to Typed Assembly Language.” *ACM TOPLAS*, vol. 21, no. 3, pp. 527–568, 1999.
- [7] A. W. Appel. “Foundational Proof-Carrying Code.” In *LICS 2001*, pp. 247–256. IEEE, 2001.
- [8] Mathlib4 Community. “mathlib4: Mathematical Library for Lean 4.” <https://github.com/leanprover-community/mathlib4>
- [9] K. Yang, G. Poesia, J. Liu, and N. D. Goodman. “LeanDojo: Theorem Proving with Retrieval-Augmented Language Models.” In *NeurIPS 2023 Datasets and Benchmarks Track*, 2023.
- [10] L. Mikula, S. Zhao, W. Nawrocki, and J. Fleuriot. “LeanHammer: Leveraging Automated Theorem Provers for Lean 4.” In *CPP 2025*, 2025.
- [11] J. Benet. “IPFS—Content Addressed, Versioned, P2P File System.” arXiv:1407.3561, 2014.
- [12] B. Laurie, A. Langley, and E. Kasper. “Certificate Transparency.” RFC 6962, IETF, 2013.
- [13] Sigstore. “Rekor: Software Supply Chain Transparency Log.” <https://github.com/sigstore/rekor>
- [14] Microsoft. “Agent Governance Toolkit.” <https://github.com/microsoft/agent-governance-toolkit>
- [15] Moonshot AI. “Kimi K2.6: 1,000-Agent Swarm Architecture.” Technical report, 2026.
- [16] Y. Bai, S. Kadavath, S. Kundu, et al. “Constitutional AI: Harmlessness from AI Feedback.” arXiv:2212.08073, 2022.
- [17] D. Song. “ASSERT-I: Assured Security and Safety through Evidence-based, Resilient, and Trustworthy Intelligence.” DARPA Information Innovation Office, 2025.
- [18] S. A. Seshia, D. Sadigh, and S. S. Sastry. “Toward Verified Artificial Intelligence.” *Communications of the ACM*, vol. 65, no. 7, pp. 46–55, 2022.

A Architecture Diagram



Vassilev mapping:

- Constitutional Halt Gate + Receipt Chain → Continuous monitoring + update cycle
- Agent Counsel Colony (Byzantine + SIFT) → Proactive red-teaming
- Human Gate + Open-Obligation Surface → Resilience mechanisms

B Phenomenological Receipt

The receipt chain is not limited to formal proofs. It receipts everything—including literary artifacts—into the same content-addressed chain. The phenomenological receipt (May 2026) demonstrates this unification:

- **Phenomenological measurements** (40ms temporal units, 2.3mm fusiform radius, 64-channel EEG) are formally mapped to technical parameters (64 attention heads, LCI order parameter, $N_c = 3.41 \times 10^{10}$)
- **Closure condition:** $D(\text{phrase}) = \text{phrase}$ —the receipt is self-referential and acknowledges it
- **Receipt anchor:** The phenomenological receipt unifies literary and formal work under a single non-repudiation framework
- **Significance:** The chain anchors any artifact—mathematical proof, literary text, phenomenological measurement—to one cryptographic root. This is the broadest instantiation of the “receipt-first” doctrine.